

# How does a neural network learn?

$$x \longrightarrow f_{\theta}(x) \longrightarrow \hat{y} \longrightarrow L \longrightarrow \nabla_{\theta} L \longrightarrow \theta'$$

# Supervised learning: data, model and prediction

## 1. Data: observed input-output pairs

Training dataset

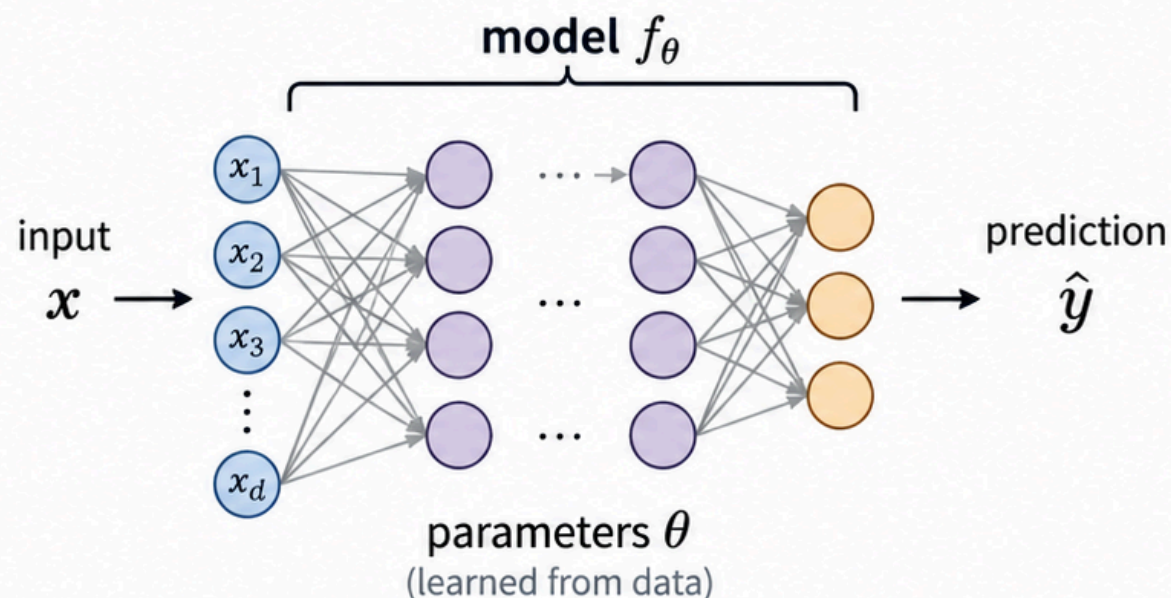
$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

| $x_1$ | $x_2$ | $x_3$ | ... | $y$ |
|-------|-------|-------|-----|-----|
| 0.5   | 1.2   | 3.1   | ... | 0   |
| 2.1   | 0.1   | 0.3   | ... | 1   |
| 1.0   | 0.8   | 2.2   | ... | 0   |
| 0.3   | 1.5   | 1.1   | ... | 1   |
| ...   | ...   | ...   | ... | ... |



## 2. Model: learn a function that maps $x$ to $y$

$$\hat{y} = f_{\theta}(x)$$



### Examples

- **Classification:**  $\hat{y} = f_{\theta}(x) \in \{0, 1, \dots, K - 1\}$
- **Regression:**  $\hat{y} = f_{\theta}(x) \in \mathbb{R}$

# We observe data

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

We start from **observed examples**, not from explicit rules.

# From data to a learning problem

## 1. Real-world problem



We observe inputs  $x$   
from the real world  
(e.g., images, text, signals, tables).

## 2. Dataset and splits

### Labeled dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$



$(x_1, y_1)$   
 $y_1 = \text{cat}$



$(x_2, y_2)$   
 $y_2 = \text{dog}$



$(x_3, y_3)$   
 $y_3 = \text{bird}$

$\vdots$

### Training set

$\mathcal{D}_{train}$   
(e.g., 70–80%)

### Validation set

$\mathcal{D}_{val}$   
(e.g., 10–15%)

### Test set

$\mathcal{D}_{test}$   
(e.g., 10–15%)

## 3. Learning problem

### Given

- a dataset  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- a model  $f_{\theta}(x)$
- a loss function  $\mathcal{L}(y, \hat{y})$

### Learn parameters

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(x_i))$$

### Use the learned model to predict

$$\hat{y} = f_{\theta^*}(x)$$

### Examples

- **Image classification:**  $\hat{y} \in \{0, 1, \dots, K-1\}$
- **Regression:**  $\hat{y} \in \mathbb{R}$
- **Multi-label:**  $\hat{y} \in \{0, 1\}^K$
- **Sequence prediction:**  $\hat{y} = (y_1, \dots, y_T)$

# The input

$x$

The information available **before** making a prediction.

- tabular attributes
- images
- sequences
- signals

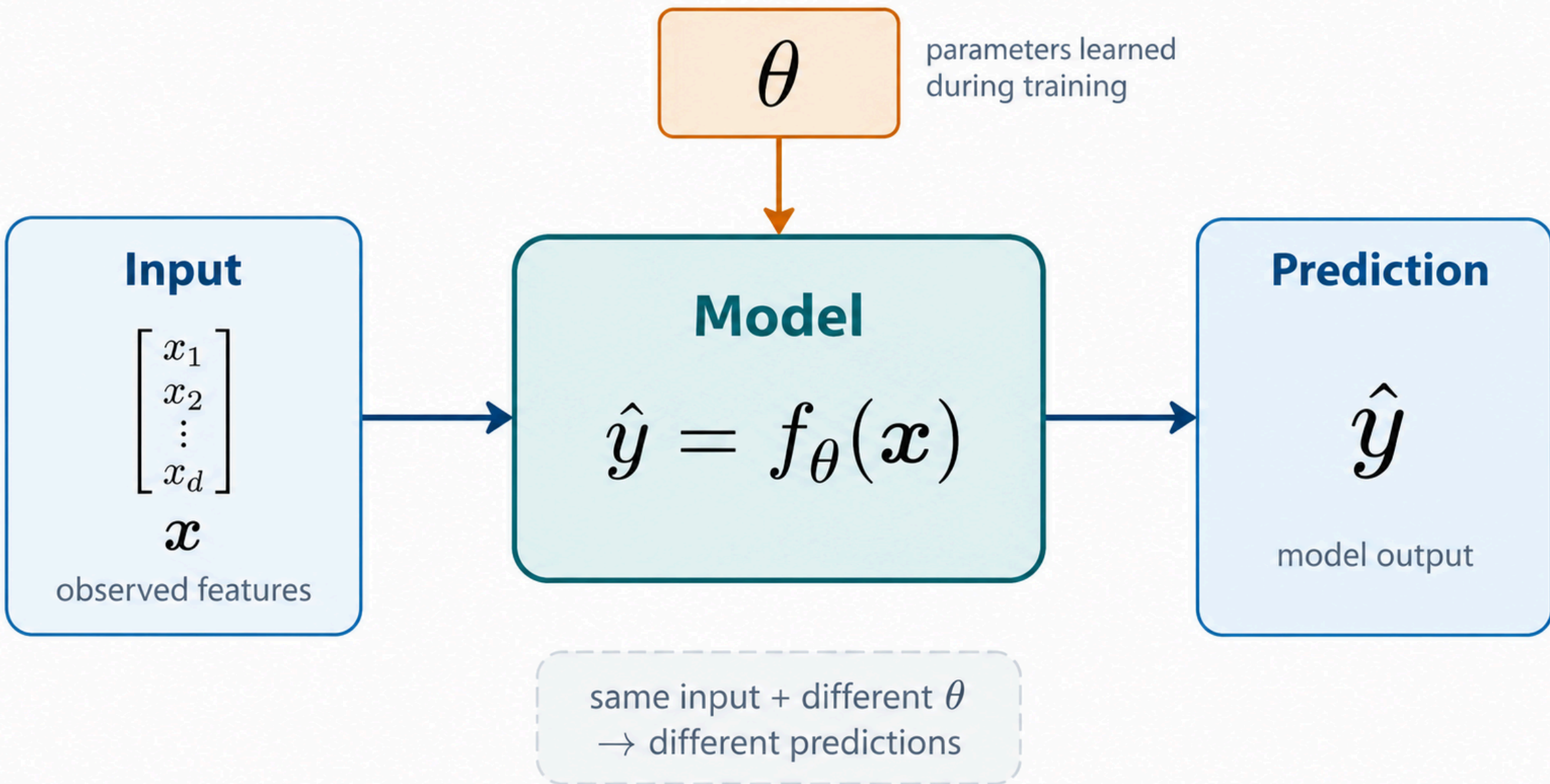
# The target

$y$

The value or class we want to predict.

$(x, y)$

The target is combined with the prediction to compute the loss during training.



# Prediction

$$\hat{y} = f_{\theta}(x)$$

Inference means using the **current parameters** to produce an output.

# Is the prediction good?

$y$

$\hat{y}$

How do we transform the quality of a prediction into something we can optimize?

# Loss

$$L(y, \hat{y})$$

For regression, one possibility is:

$$L = (y - \hat{y})^2$$

A loss converts the task objective into a **scalar quantity**.

# Loss depends on the parameters

$$\hat{y} = f_{\theta}(x)$$

therefore,

$$L(y, \hat{y}) = L(y, f_{\theta}(x))$$

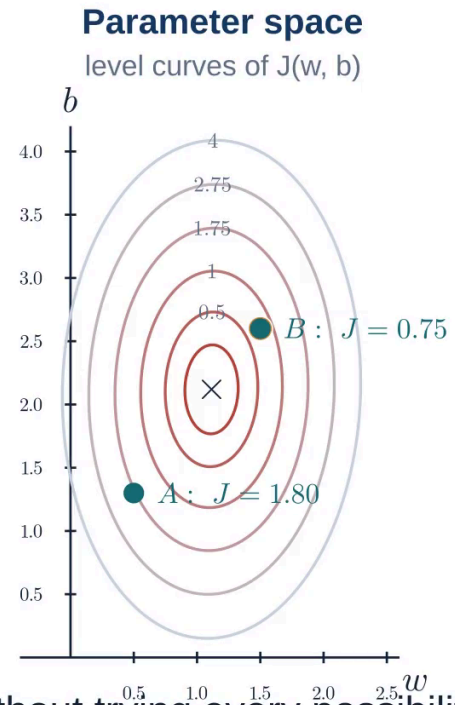
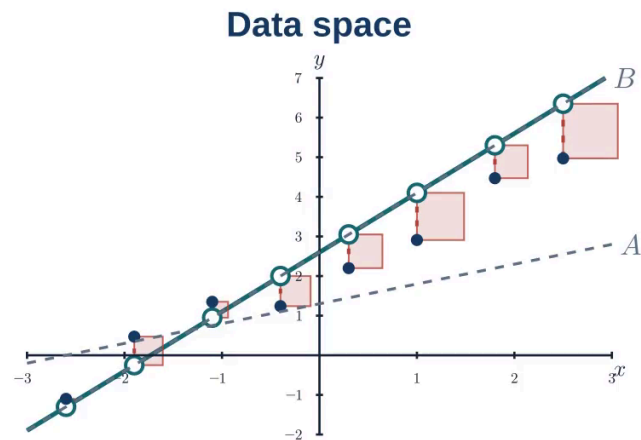
and we can reason about:

$$L(\theta)$$

# Parameters, residuals and loss

## Parameters, residuals and loss

$$w = 1.50 \quad b = 2.60 \quad \text{MSE} = 0.75$$

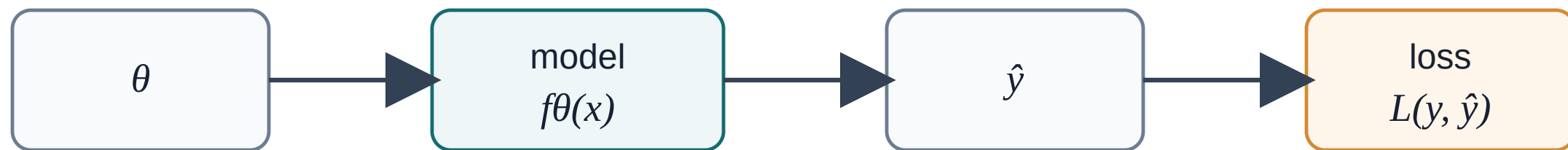


**Question:** how do we find a better configuration without trying every possibility?

[Open animation](#)

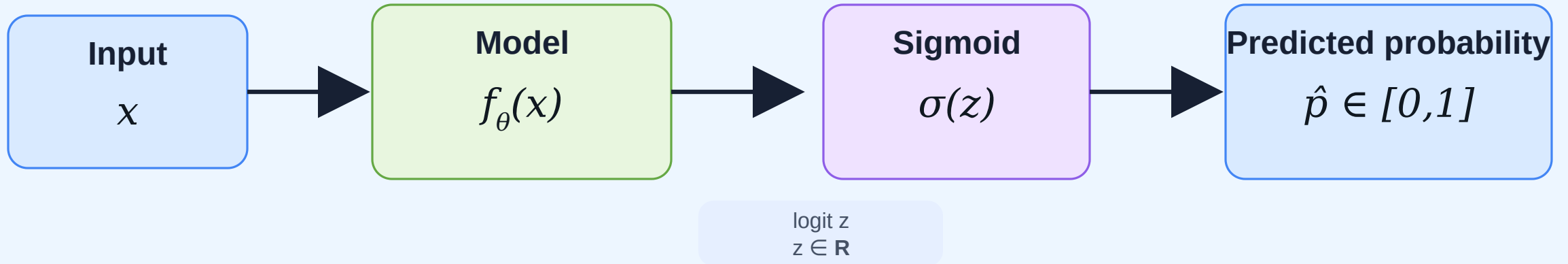
# How parameters determine the loss

The loss depends on  $\theta$  indirectly through the model prediction.



the gradient communicates how the loss changes with respect to  $\theta$

# Binary classification: logits, sigmoid and BCE



## Binary cross-entropy

$$z = f_{\theta}(x)$$

$$\hat{p} = \sigma(z) = 1 / (1 + e^{-z})$$

$$L = -[ y \log(\hat{p}) + (1-y) \log(1-\hat{p}) ]$$

$y \in \{0,1\}$

**PyTorch**

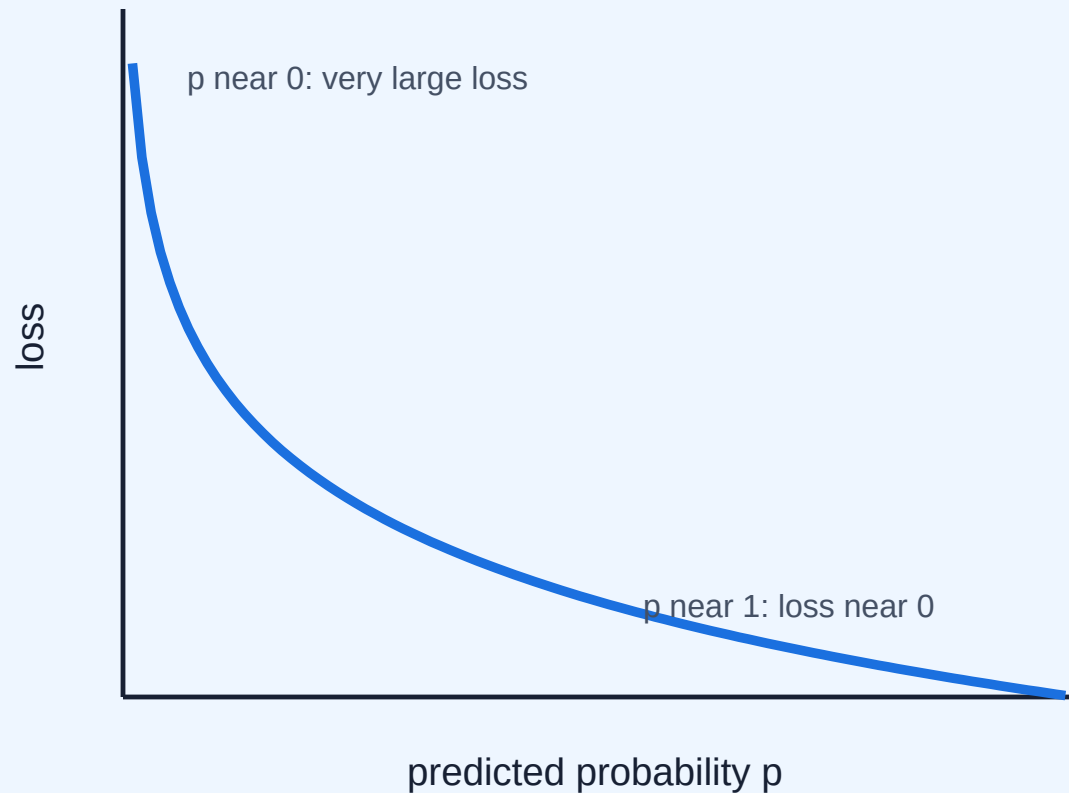
`nn.BCEWithLogitsLoss`

expects logits  $z$ ; sigmoid is applied internally.

# Binary cross-entropy: the typical loss curves

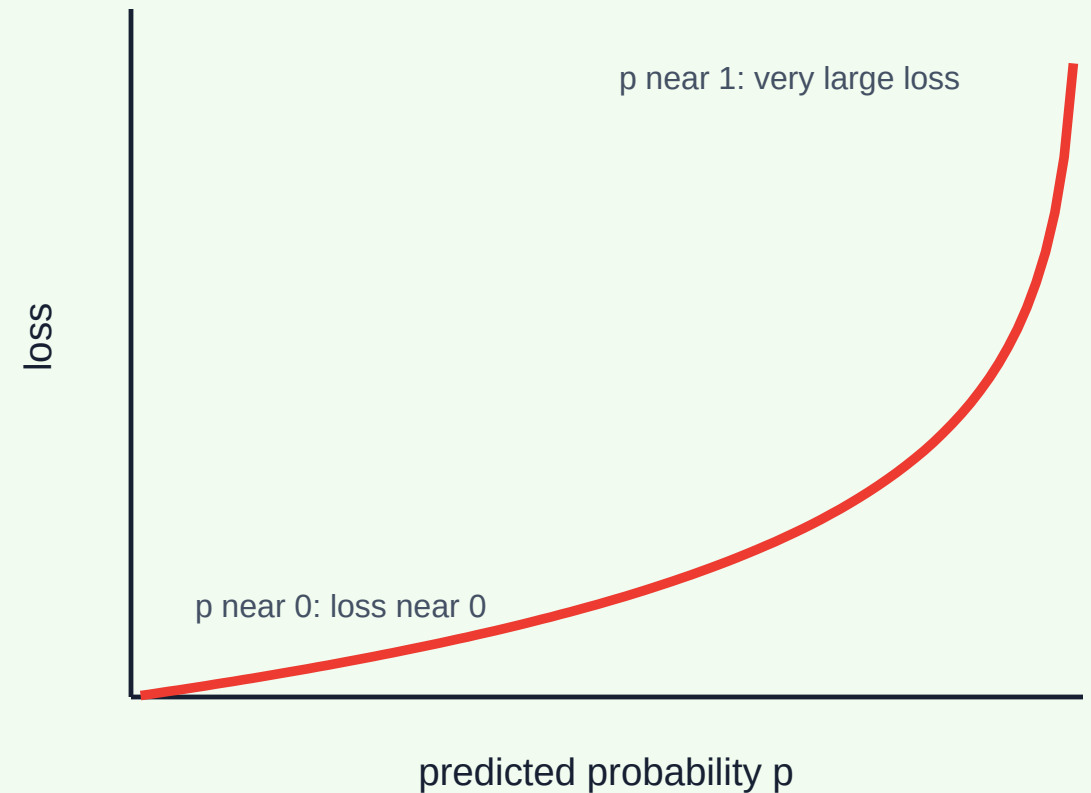
True label  $y = 1$

$$L(p, 1) = -\log(p)$$



True label  $y = 0$

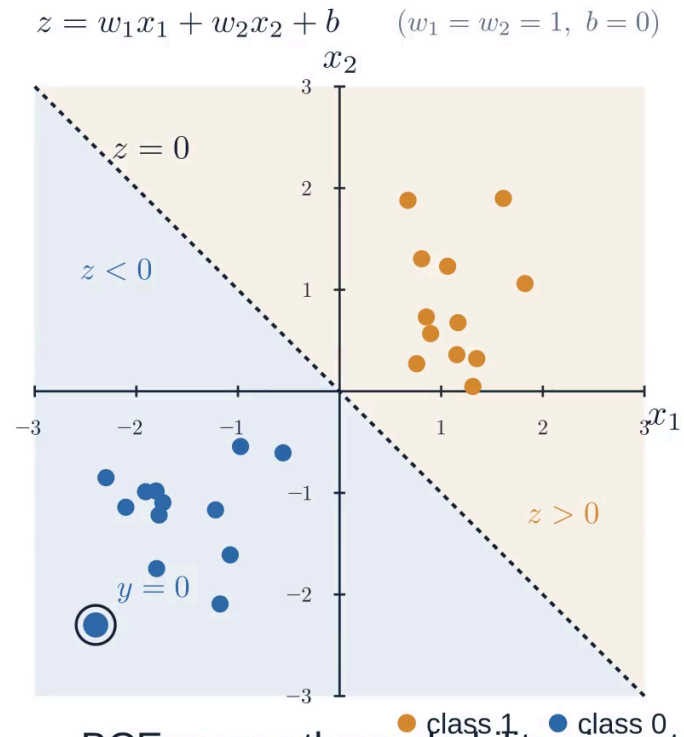
$$L(p, 0) = -\log(1-p)$$



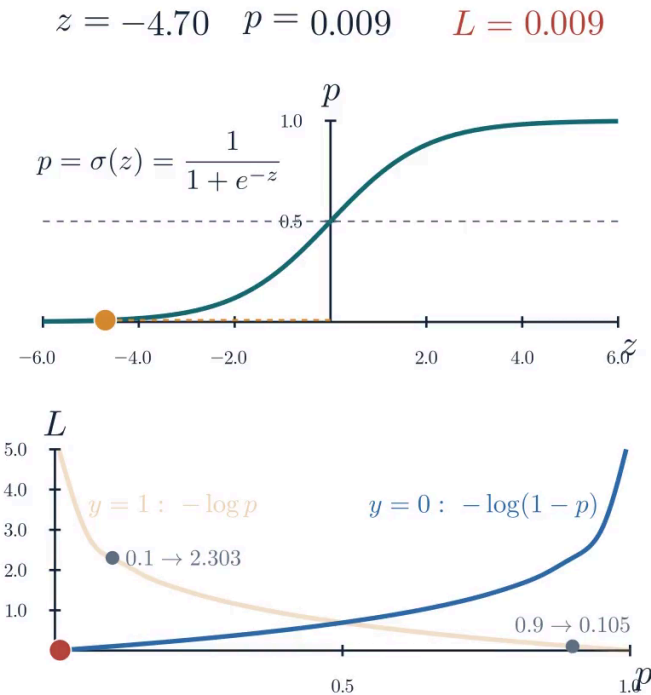
**Confident wrong predictions receive a very large penalty.**

# From score to probability to loss

## From score $z$ to probability to loss

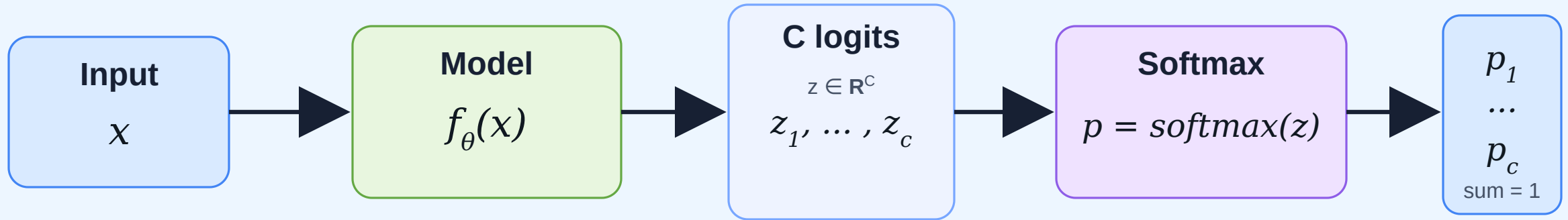


BCE scores the probability given to the true class:  $p$  for  $y = 1$ ,  $1 - p$  for  $y = 0$ .



[Open animation](#)

# Multiclass classification: logits, softmax and cross-entropy



## Cross-entropy for true class $y$

$$p_c = e^{z_c} / \sum_k e^{z_k}$$

$$L(z, y) = -\log p_y$$

Raise the true-class logit relative to the others.

**PyTorch**

`nn.CrossEntropyLoss`

expects raw logits (B,C) and class indices 0...(C-1).

# Cross-entropy intuition: confidence matters

## Binary classification

True label  $y = 1$

| Prediction | Interpretation           | Loss  |
|------------|--------------------------|-------|
| $p = 0.9$  | correct, high confidence | 0.105 |
| $p = 0.6$  | correct, low confidence  | 0.511 |
| $p = 0.1$  | wrong, high confidence   | 2.303 |

## Multiclass classification

True class  $y = 2$

| Predicted probabilities | Interpretation             | Loss  |
|-------------------------|----------------------------|-------|
| $[0.1, 0.8, 0.1]$       | correct<br>high confidence | 0.223 |
| $[0.2, 0.5, 0.3]$       | correct<br>low confidence  | 0.693 |
| $[0.7, 0.2, 0.1]$       | wrong<br>high confidence   | 1.609 |

More probability on the true class means lower cross-entropy.

# What the loss function expects in PyTorch

| Task                      | Model output                                    | Target                    | Loss in PyTorch                                                        | Shapes                                                      |
|---------------------------|-------------------------------------------------|---------------------------|------------------------------------------------------------------------|-------------------------------------------------------------|
| Regression                | $\hat{y} \in \mathbf{R}$                        | $y \in \mathbf{R}$        | <code>nn.MSELoss</code>                                                | output: (B,) or (B,1)<br>target: same shape<br>dtype: float |
| Binary classification     | $z \in \mathbf{R}$<br>$\hat{p} = \sigma(z)$     | $y \in \{0,1\}$           | <code>nn.BCEWithLogitsLoss</code><br>expects logits, not probabilities | logits: (B,) or (B,1)<br>target: same shape<br>dtype: float |
| Multiclass classification | $z \in \mathbf{R}^C$<br>$p = \text{softmax}(z)$ | $y \in \{0, \dots, C-1\}$ | <code>nn.CrossEntropyLoss</code><br>expects logits and class indices   | logits: (B,C)<br>target: (B,)<br>dtype: long                |

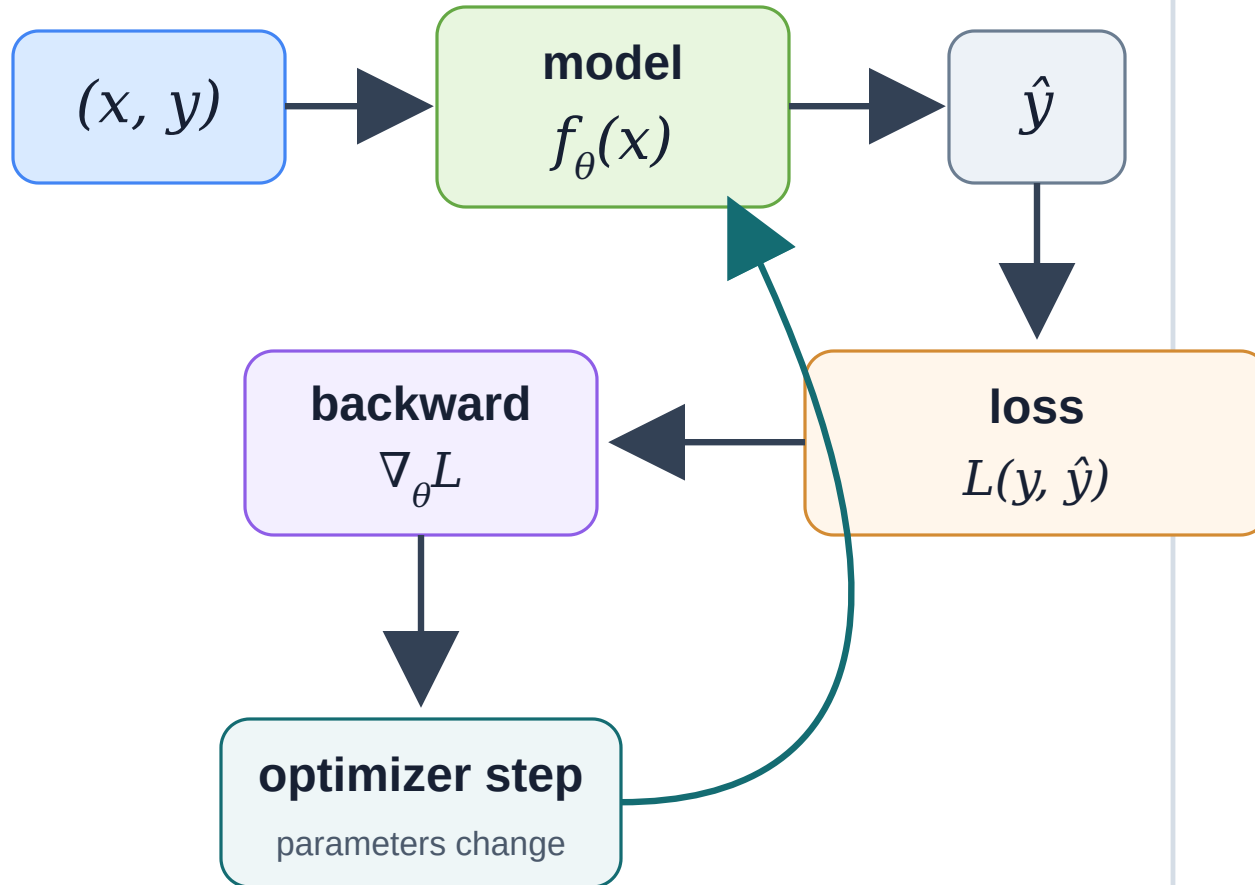
Use sigmoid/softmax to interpret predictions; train with raw logits.

# Training is not inference

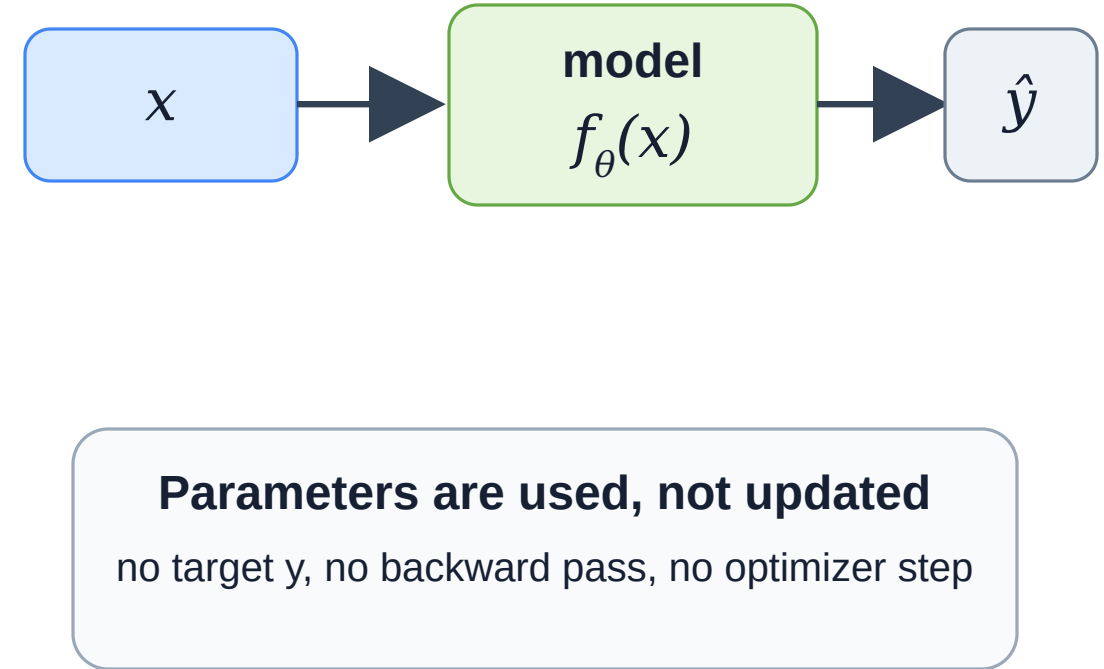
| Inference                                         | Training                                                                                 |
|---------------------------------------------------|------------------------------------------------------------------------------------------|
| $x \rightarrow f_{\theta}(x) \rightarrow \hat{y}$ | $(x, y) \rightarrow f_{\theta}(x) \rightarrow \hat{y} \rightarrow L \rightarrow \theta'$ |
| parameters are <b>used</b>                        | parameters are <b>changed</b>                                                            |

# Training versus inference

## Training



## Inference



# One parameter

Suppose:

$$\hat{y} = wx$$

Changing  $w$  changes the prediction and therefore changes the loss.

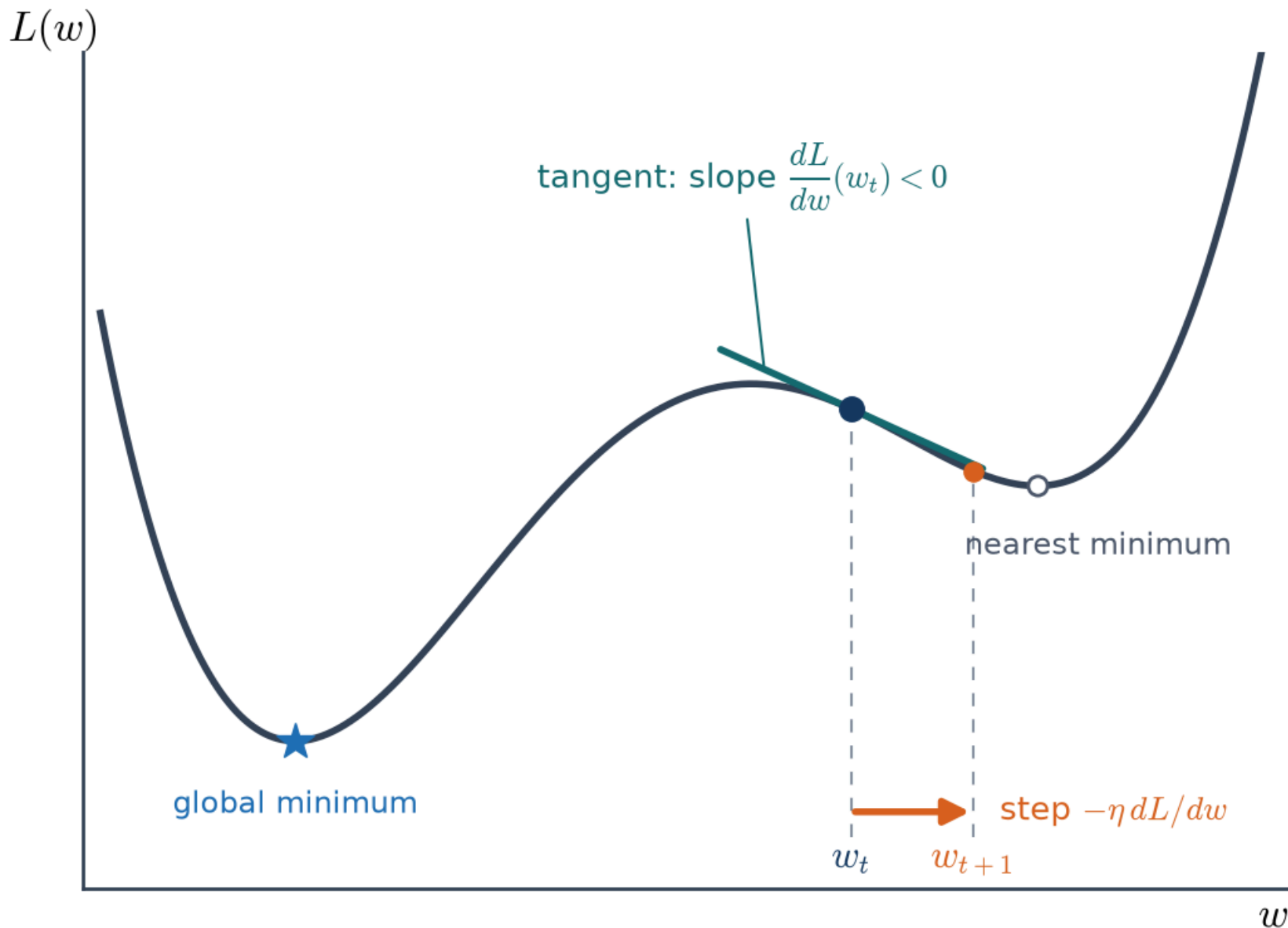
# The gradient

$$\frac{\partial L}{\partial w}$$

The gradient gives **local information** about how the loss changes.

# The derivative is local information

At the current value  $w_t$ , the slope says how  $L$  changes nearby, and nothing more.



## What the gradient tells us

- the direction in which  $L$  decreases at  $w_t$
- how steep it is there

## What it does not tell us

- where the minimum is
- whether a better minimum exists elsewhere

# Gradient descent

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}$$

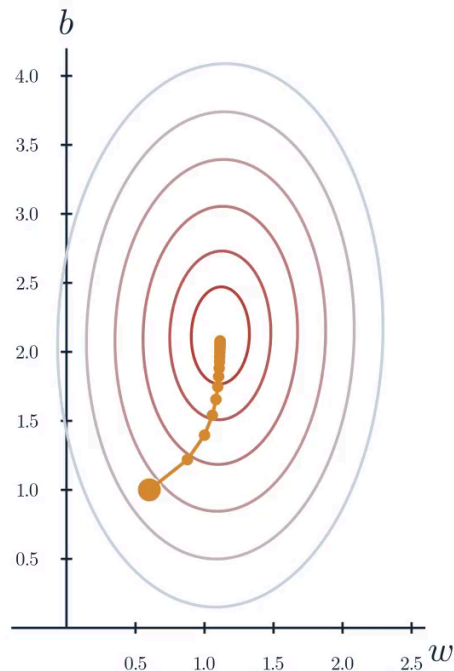
Two ingredients:

- **direction** from the gradient
- **step size** from the learning rate

# Derivative, gradient and update

## Derivative, gradient and update

MSE of the regression above: level curves of  $J(w, b)$



$$\nabla J(w, b) = \begin{bmatrix} \partial J / \partial w \\ \partial J / \partial b \end{bmatrix}$$

$$\nabla J(0.6, 1.0) \approx \begin{bmatrix} -2.76 \\ -2.19 \end{bmatrix}$$

Arrows have fixed length: they show direction only.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t), \quad \eta = 0.1$$

$$t = 15 \quad J(\theta_t) = 0.13$$

Each step uses the gradient at the current point. The path bends toward the minimum.

[Open animation](#)

# Learning rate

$\eta$

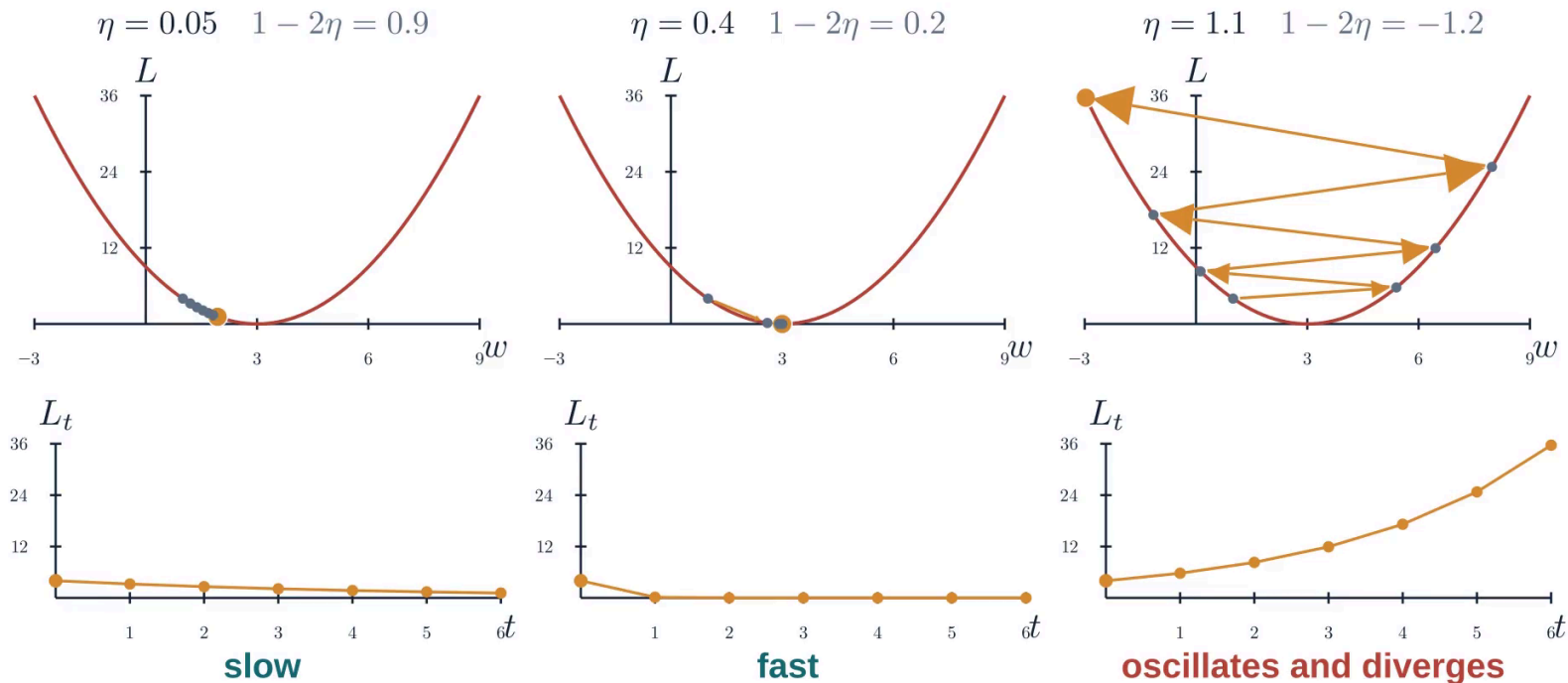
What changes if the step is:

- too small?
- appropriate?
- too large?

# Comparing learning rates

## Learning rate

$$w_{t+1} - 3 = (1 - 2\eta)(w_t - 3), \quad w_0 = 1 \quad t = 6$$



Here it converges if  $0 < \eta < 1$ . These values belong to this function and scale; they are not a recipe for networks.

[Open animation](#)

## More than one parameter

$$\theta = (w_1, w_2, \dots, w_p)$$

$$\nabla_{\theta} L = \begin{bmatrix} \partial L / \partial w_1 \\ \vdots \\ \partial L / \partial w_p \end{bmatrix}$$

# From example loss to dataset objective

For one observation:

$$L_i(\theta) = L(y_i, f_\theta(x_i))$$

For the full dataset:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta)$$

**Convention:**  $L$  is a per-example loss;  $J$  is the dataset objective.

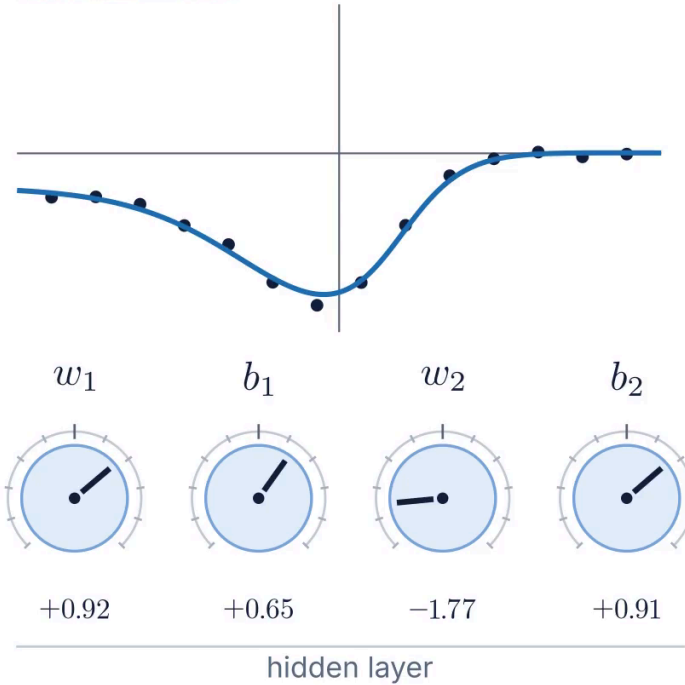
# Each parameter is a knob

## Each parameter is a knob

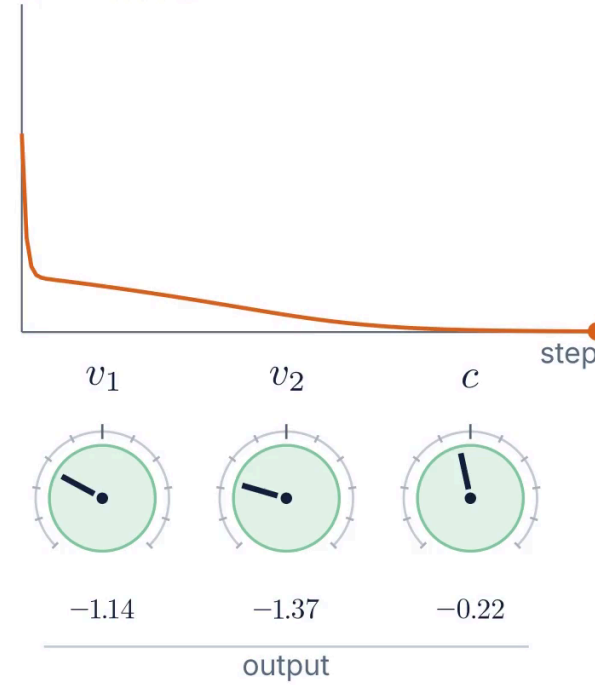
$$f(x) = v_1 \tanh(w_1 x + b_1) + v_2 \tanh(w_2 x + b_2) + c$$

At the minimum  $\nabla_{\theta} J \approx 0$  : there is nowhere left to turn.

data and model



$J = 0.004$



[Open animation](#)

# Batch, SGD and mini-batch

For a subset  $\mathcal{B}$  of the data:

$$J_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} L_i(\theta)$$

- **batch:**  $\mathcal{B} = \mathcal{D}$
- **SGD:**  $|\mathcal{B}| = 1$
- **mini-batch:**  $1 < |\mathcal{B}| < N$

The update uses  $g_t = \nabla_{\theta} J_{\mathcal{B}_t}(\theta_t)$ .

$$J_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \mathcal{L}_i(\theta)$$

$B$  = subset used to estimate the gradient at step  $t$

## Batch gradient descent

Dataset  $\mathcal{D}$  ( $N$  examples)



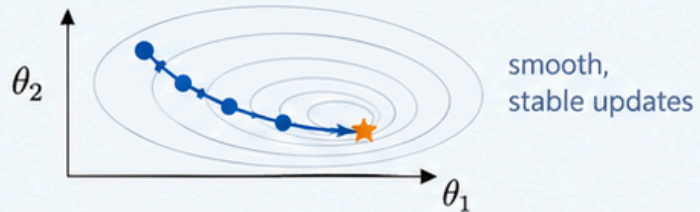
$$B = \mathcal{D}$$

(use all  $N$  examples)

$$g_t = \nabla_{\theta} J_{\mathcal{D}}(\theta)$$

uses all examples  
before each update

- low gradient noise
- high computation per step



## Stochastic gradient descent (SGD)

Dataset  $\mathcal{D}$  ( $N$  examples)



selected example  $i$  at step  $t$

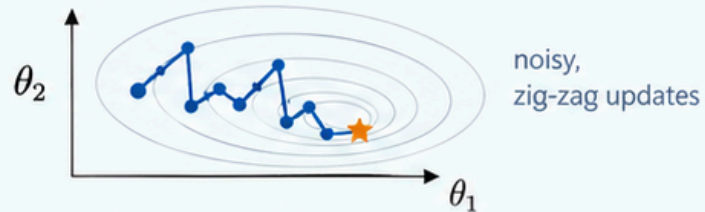
$$|B| = 1$$

(one example)

$$g_t = \nabla_{\theta} \mathcal{L}_i(\theta)$$

uses one example  
per update

- high gradient noise
- cheap updates



## Mini-batch gradient descent

Dataset  $\mathcal{D}$  ( $N$  examples)



mini-batch  $B_t$  at step  $t$

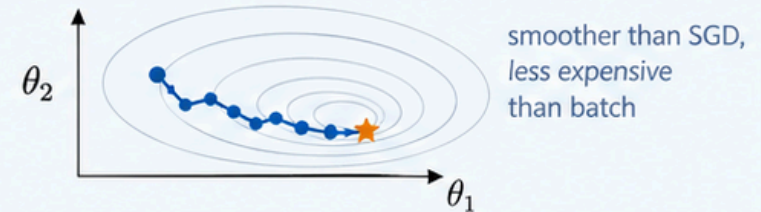
$$1 < |B| < N$$

(small subset)

$$g_t = \nabla_{\theta} J_{B_t}(\theta)$$

uses a small subset  
per update

- balanced noise / cost
- GPU-friendly



**Batch = accurate but expensive** • **SGD = noisy but fast** • **Mini-batch = practical default in deep learning**

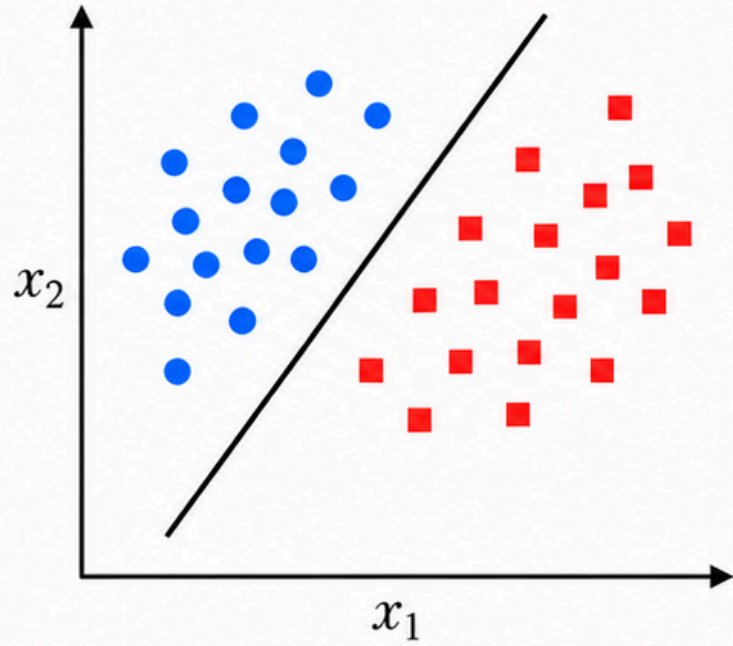
# Optimization is a trajectory

$$\theta_{t+1} = \theta_t - \eta g_t$$
$$\theta_0, \theta_1, \theta_2, \dots$$

Training moves through parameter space using successive gradient estimates.

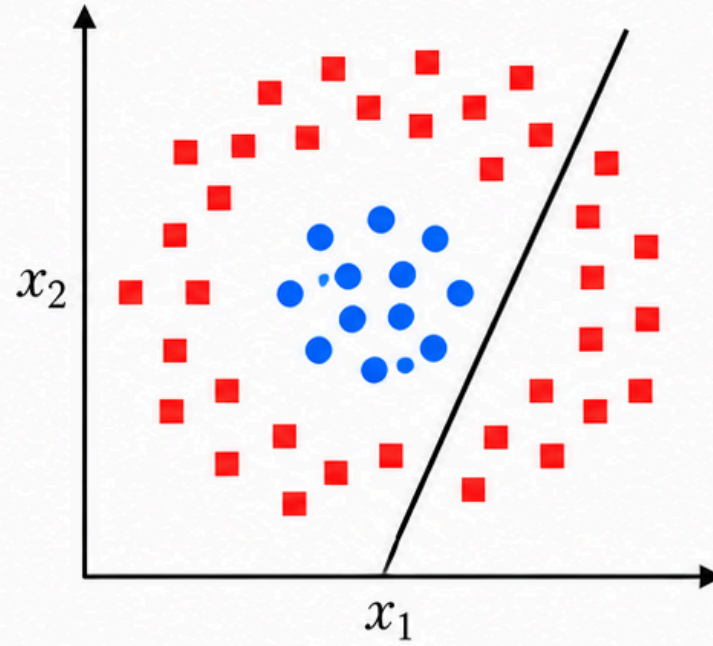
### Linearly separable data

A linear model is sufficient.



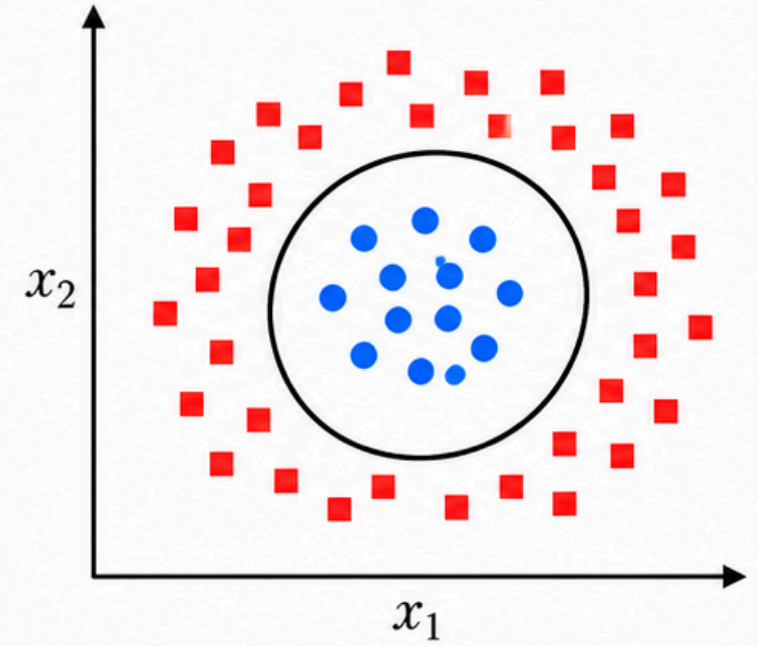
### Non-linearly separable data

A linear model cannot represent the boundary.



### Non-linear model

Hidden layers + nonlinearity can fit the structure.



**Optimization can only fit functions that the model class can represent.**

# One artificial neuron

An input  $x \in \mathbb{R}^d$ , one weight vector  $w \in \mathbb{R}^d$  and one bias  $b \in \mathbb{R}$ :

$$z = w^\top x + b = \sum_{j=1}^d w_j x_j + b, \quad a = \phi(z)$$

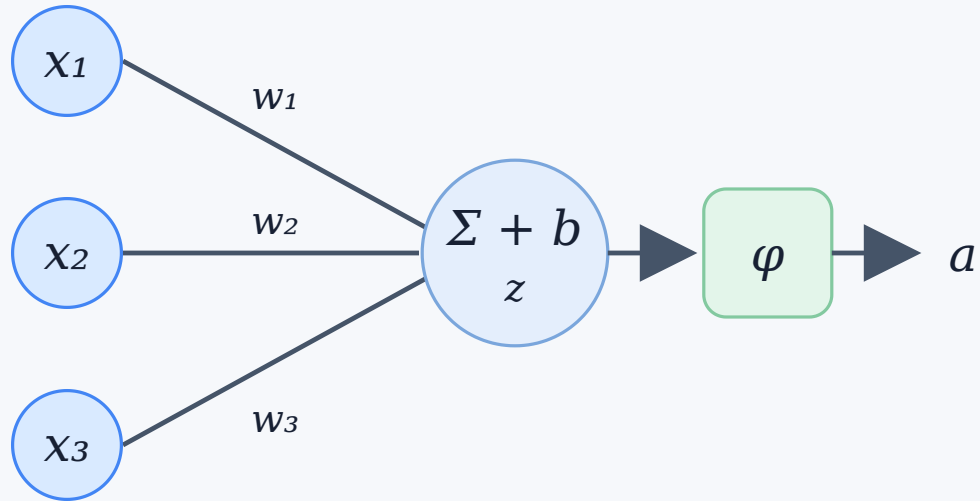
- $z$ : a **score** (weighted sum of the inputs plus a shift)
- $\phi$ : an activation applied to that score

We have already used this neuron:  $\phi(z) = z$  is **linear regression**;  $\phi = \sigma$  is **logistic regression**.

# From one neuron to a layer

Each neuron has its own weight vector and bias; a layer stacks them as the rows of  $W$ .

One neuron

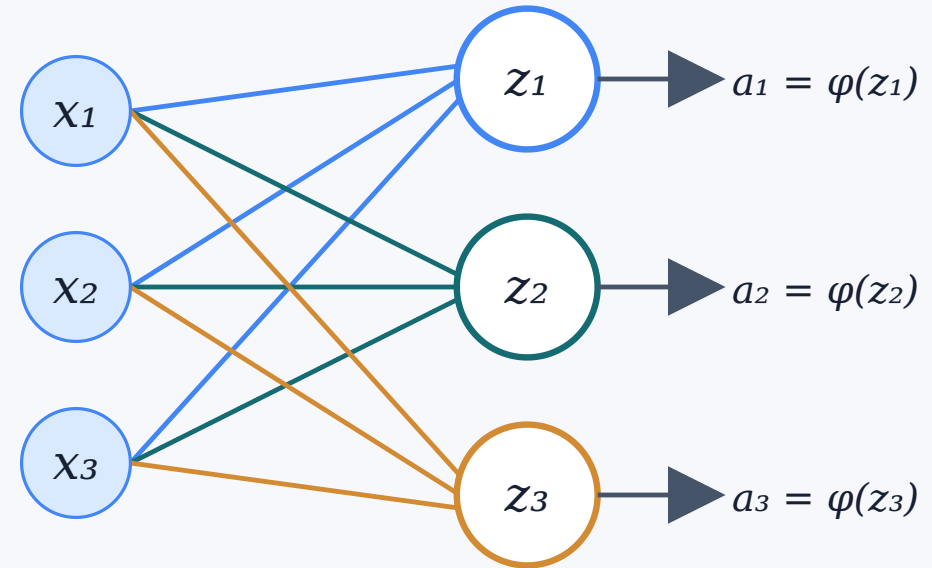


$$z = w^T x + b = \sum_j w_j x_j + b$$

$$a = \phi(z)$$

$w \in \mathbb{R}^3, b \in \mathbb{R} \rightarrow$  one score  $z$ , one output  $a$

A layer of  $m = 3$  neurons



$$W = \begin{bmatrix} w_1^T \\ w_2^T \\ w_3^T \end{bmatrix} \in \mathbb{R}^{3 \times 3}, b \in \mathbb{R}^3$$

$$z = Wx + b$$

$$a = \phi(z)$$

$\phi$  acts elementwise

# From one neuron to a layer

$m$  neurons read the **same** input, each with its own weights and bias:

$$z_k = w_k^\top x + b_k, \quad k = 1, \dots, m$$

Stacking the  $m$  equations:

$$\underbrace{\begin{bmatrix} z_1 \\ \vdots \\ z_m \end{bmatrix}}_z = \underbrace{\begin{bmatrix} w_1^\top \\ \vdots \\ w_m^\top \end{bmatrix}}_W x + \underbrace{\begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}}_b, \quad a = \phi(z) \text{ (elementwise)}$$

Row  $k$  of  $W$  is neuron  $k$ .

# Layer shapes

| Object | Shape        | Meaning                             |
|--------|--------------|-------------------------------------|
| $x$    | $d$          | one example, $d$ features           |
| $W$    | $m \times d$ | one row per neuron                  |
| $b$    | $m$          | one bias per neuron                 |
| $z, a$ | $m$          | one score and one output per neuron |

For a batch  $X \in \mathbb{R}^{B \times d}$ :  $Z = XW^\top + b \in \mathbb{R}^{B \times m}$ .

In PyTorch, `nn.Linear(d, m)` stores `weight` with shape `(m, d)` and `bias` with shape `(m, )`.

# Stacking linear transformations

$$h = W_1x + b_1$$

$$\hat{y} = W_2h + b_2$$

Therefore:

$$\hat{y} = W_2W_1x + W_2b_1 + b_2$$

Several linear layers can collapse into a single affine transformation.

# Nonlinearity changes the game

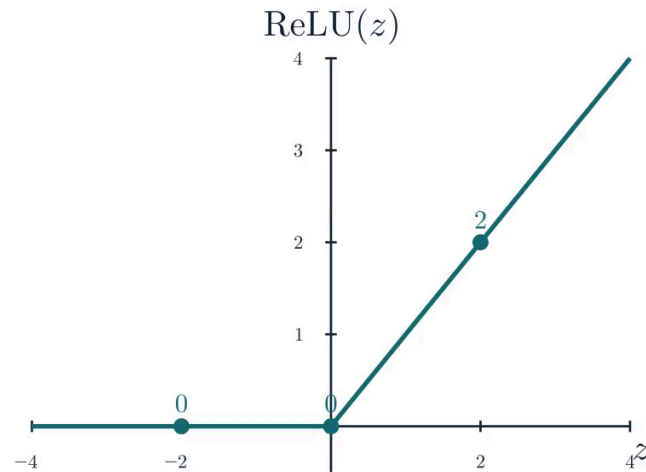
$$z = Wx + b, \quad a = \phi(z)$$

A nonlinear activation prevents the composition from collapsing into one linear map.

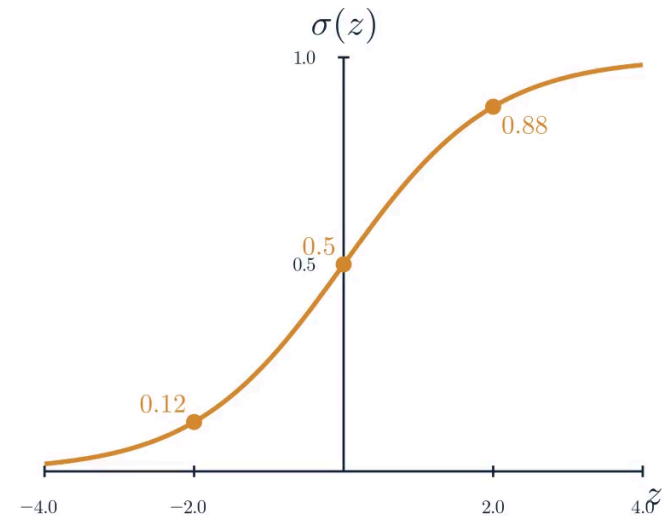
# Why we need an activation

## Why we need an activation

$\text{ReLU}(z) = \max(0, z)$   
range  $[0, \infty)$  · hidden layers



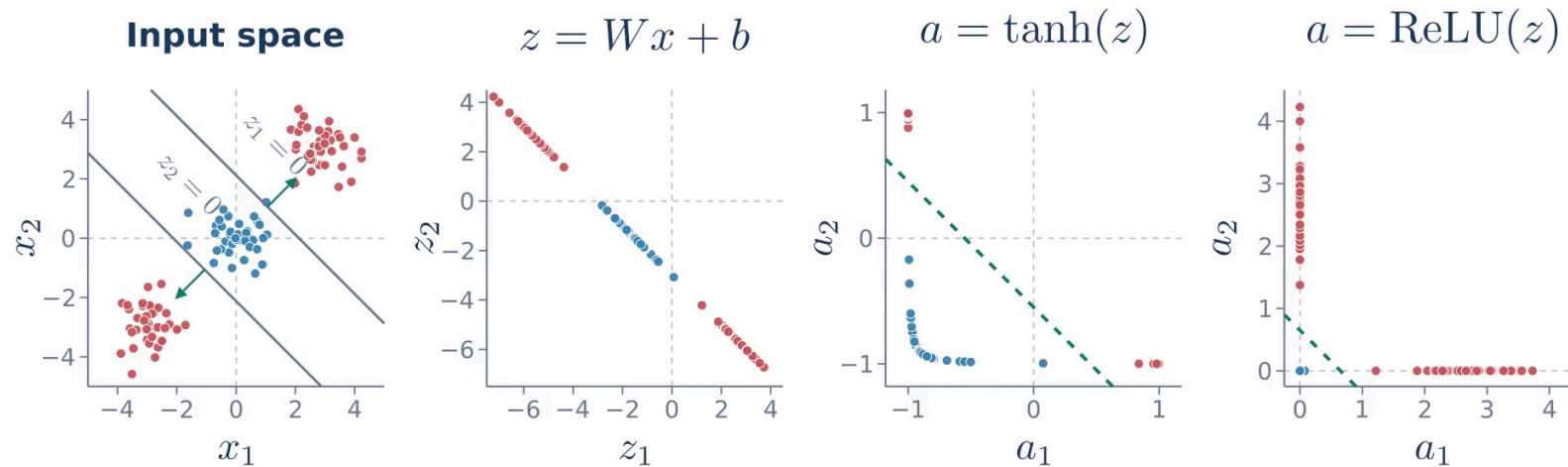
$\sigma(z) = \frac{1}{1 + e^{-z}}$   
range  $(0, 1)$  · output probability



ReLU: 0, 0, 2 (no upper bound; not differentiable at 0). Sigmoid: 0.12, 0.5, 0.88.

[Open animation](#)

# Affine maps and nonlinear activations



$$z_1 = (x_1 + x_2)/\sqrt{2} - 1.5, \quad z_2 = -(x_1 + x_2)/\sqrt{2} - 1.5$$

[Open animation](#)

# ReLU

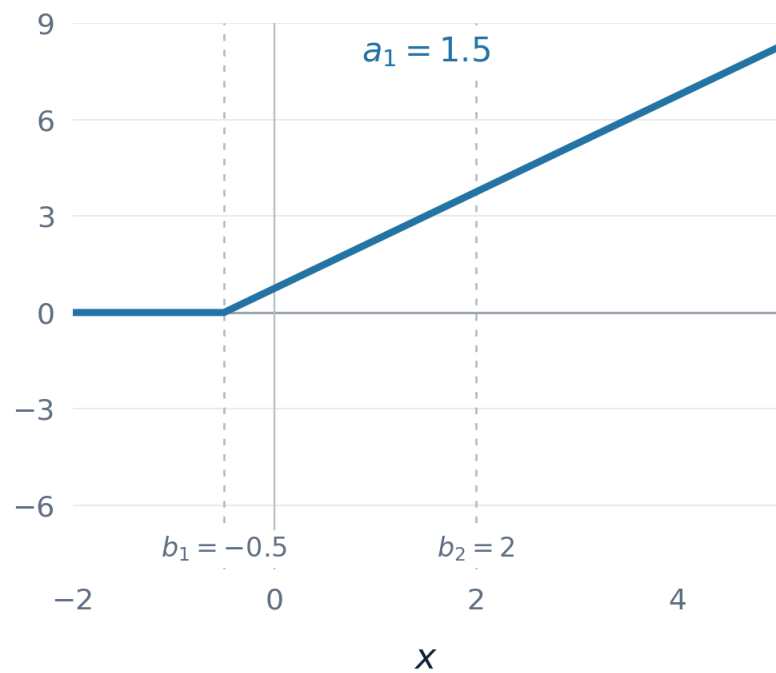
$$\text{ReLU}(z) = \max(0, z)$$

A very simple nonlinear function can become a powerful building block.

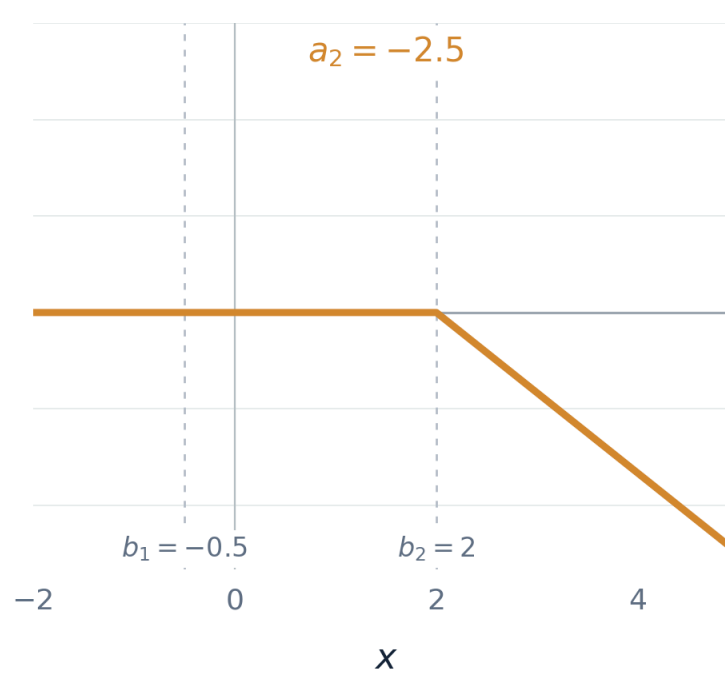
# One ReLU as a building block

A single unit produces a simple piecewise-linear transformation.

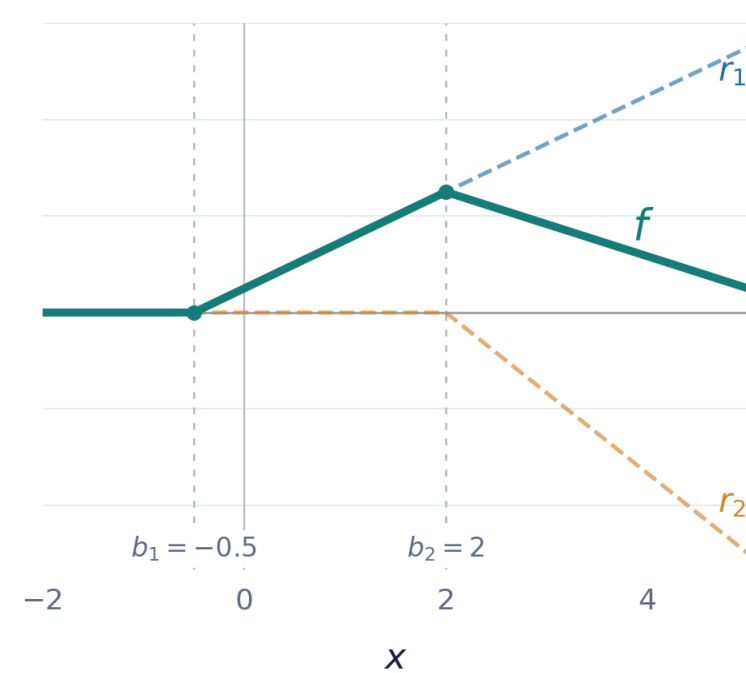
$$r_1(x) = a_1 \text{ReLU}(x - b_1)$$



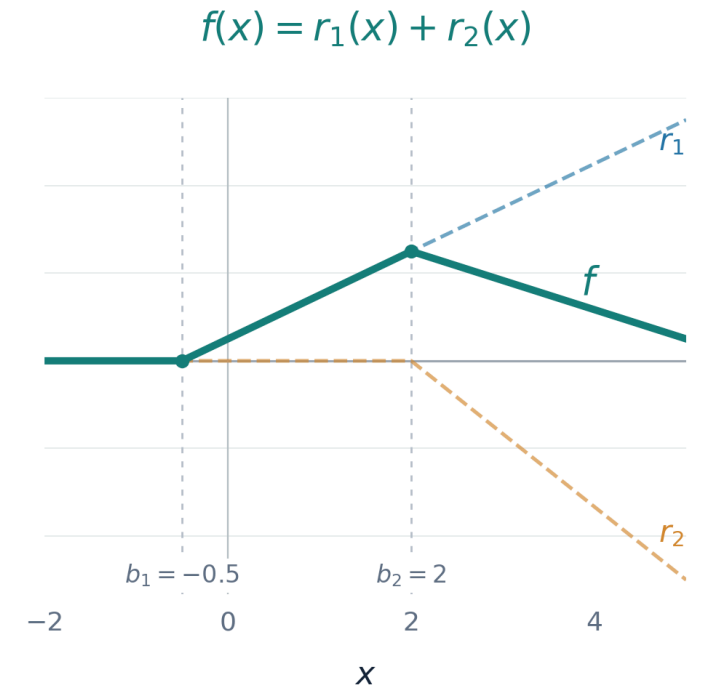
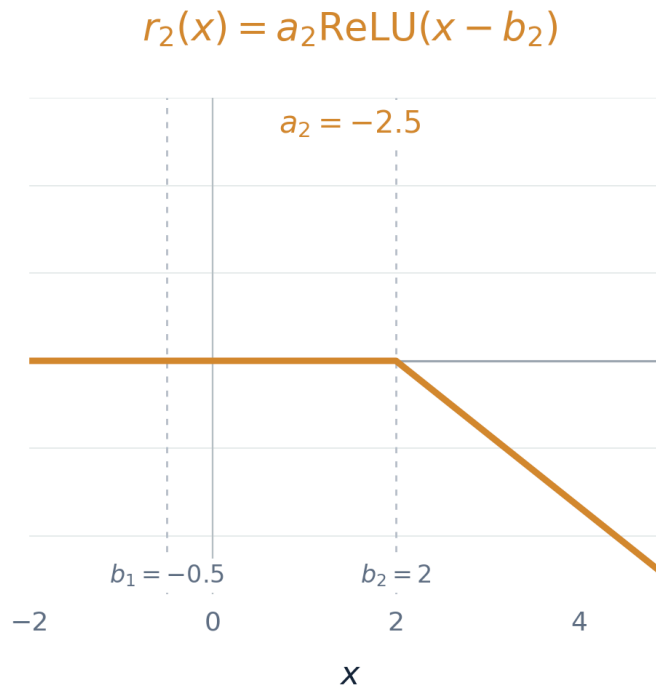
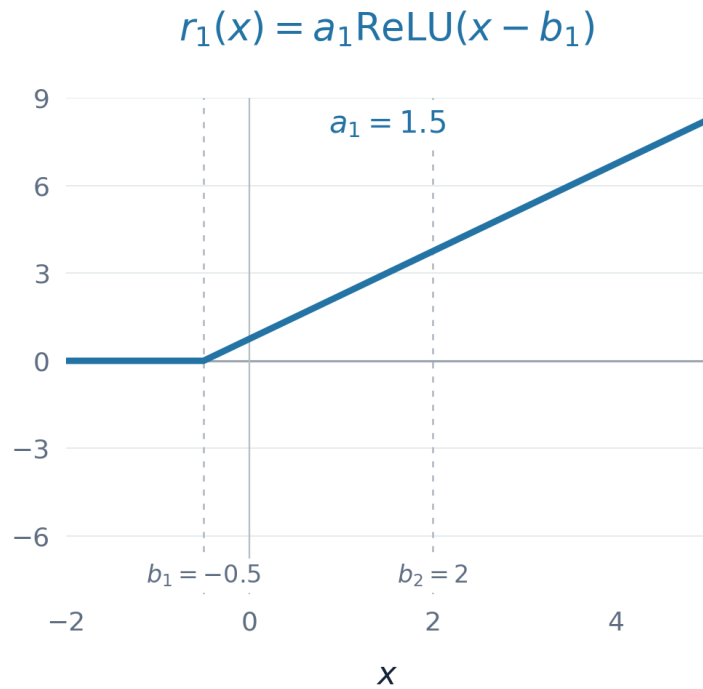
$$r_2(x) = a_2 \text{ReLU}(x - b_2)$$



$$f(x) = r_1(x) + r_2(x)$$



# Approximation with ReLU components



[Open animation](#)

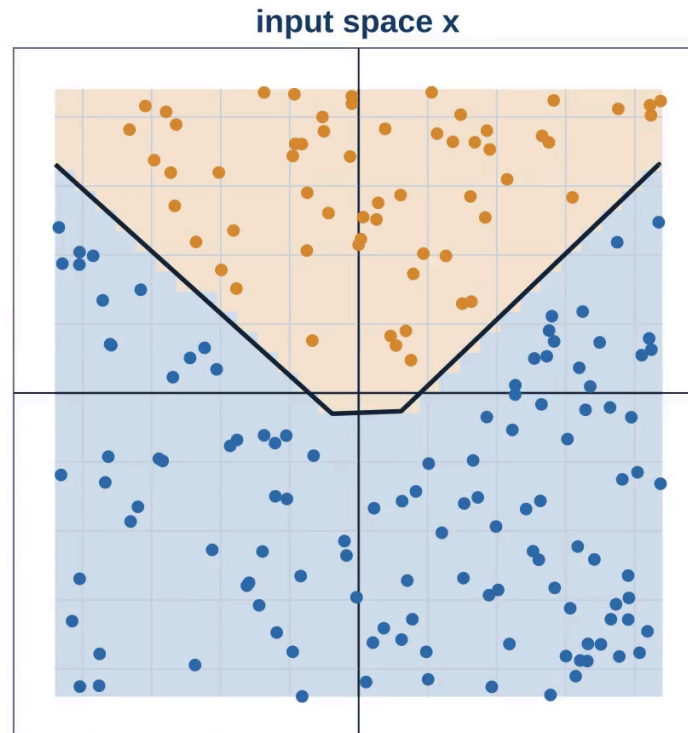
## A hidden layer

$$z_1 = W_1x + b_1, \quad a_1 = \phi(z_1)$$
$$\hat{y} = W_2a_1 + b_2$$

A hidden layer learns an **intermediate representation**.

# Hidden representation and decision boundary

## Hidden representation and decision boundary



1.  $x \in \mathbb{R}^2$
2.  $z_1 = W_1 x + b_1$
3.  $a_1 = \text{ReLU}(z_1)$
4.  $w_2^\top a_1 + b_2 = 0$
5. boundary in  $x$

Trained 2–2–1 MLP, not hand-picked weights  
numpy · seed 3 · 6000 GD steps ·  $\eta = 0.1$   
training accuracy: 100 %

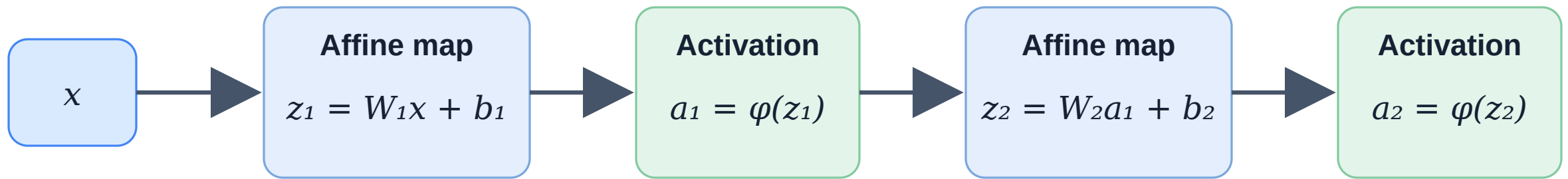
● class 1   ● class 0

With seeds 2 and 7 the same training stops at 91–93 %: capacity does not guarantee good training.

[Open animation](#)

# MLP as a composition of affine maps and activations

$\varphi$  denotes a generic nonlinear activation;  $\sigma$  is reserved for sigmoid.

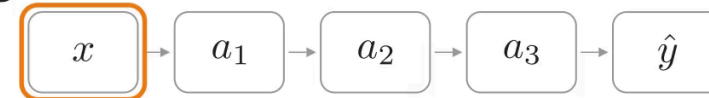
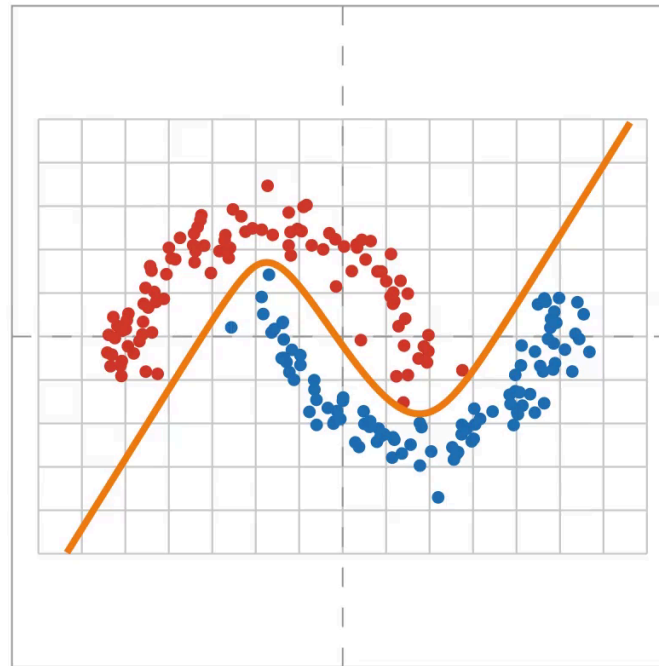


Without  $\varphi$ , stacked affine maps collapse into one affine map.

# Each layer transforms the space

## Each layer transforms the space

Trained 2-2-2-2-1 MLP · training accuracy 100%



$$\hat{y} = \sigma(w_4^\top a_3 + b_4)$$

the output layer is linear:  
one straight line in  $a_3$

The same line, seen in  $x$ ,  
is a curved boundary.

Hidden layers bend the space; the output layer only draws a straight line.

[Open animation](#)

# Forward pass

$$x \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2 \rightarrow \hat{y} \rightarrow L$$

The forward pass evaluates the composed function.

# Computational graph

Each hidden layer contains an affine map and an activation:

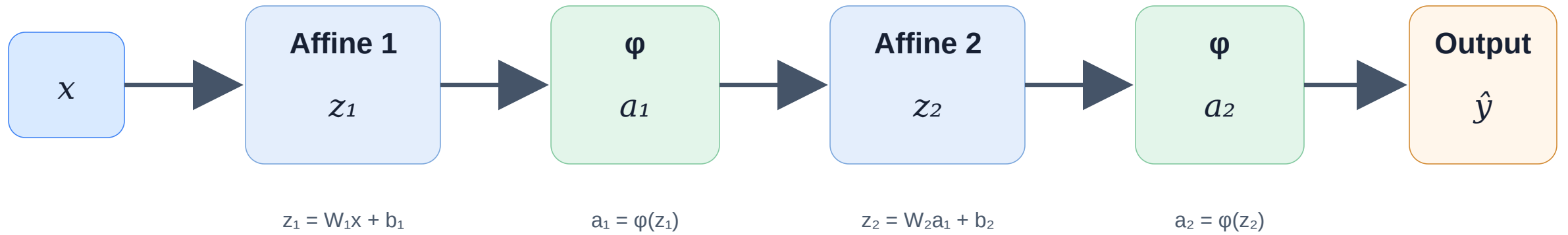
$$z_l = W_l a_{l-1} + b_l, \quad a_l = \phi(z_l), \quad a_0 = x$$

For a linear output and squared loss:

$$\hat{y} = W_3 a_2 + b_3, \quad L = \frac{1}{2} \|\hat{y} - y\|^2$$

# Forward pass through affine and nonlinear layers

Forward computes values left to right and stores intermediate values for backward.



**Saved for backward**

$x, z_1, a_1, z_2, a_2$  and  $\hat{y}$

# Chain rule

If:

$$L = L(\hat{y}(a(w)))$$

then:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a} \frac{\partial a}{\partial w}$$

# Backward pass

The same graph is traversed in the opposite direction to propagate gradients.

# Backward pass: signals and parameter gradients

For a linear output and squared loss:  $L = 1/2 ||\hat{y} - y||^2$

**Backward starts at the loss**

$$\delta_{\hat{y}} = \partial L / \partial \hat{y} = \hat{y} - y$$

seed gradient at the model output

**Through activation  $\phi$**

$$\delta_{z_1} = \delta_{a_1} \odot \phi'(z_1)$$

**Through hidden affine**

$$\delta_{a_1} = W_2^T \delta_{z_2}$$

**Through activation  $\phi$**

$$\delta_{z_2} = \delta_{a_2} \odot \phi'(z_2)$$

**Through output layer**

$$\delta_{a_2} = W_3^T \delta_{\hat{y}}$$

**Parameter gradients from the same backward signals**

**Output layer**

$$\nabla_{W_3} L = \delta_{\hat{y}} a_2^T$$

$$\nabla_{b_3} L = \delta_{\hat{y}}$$

**Hidden layer 2**

$$\nabla_{W_2} L = \delta_{z_2} a_1^T$$

$$\nabla_{b_2} L = \delta_{z_2}$$

**Hidden layer 1**

$$\nabla_{W_1} L = \delta_{z_1} x^T$$

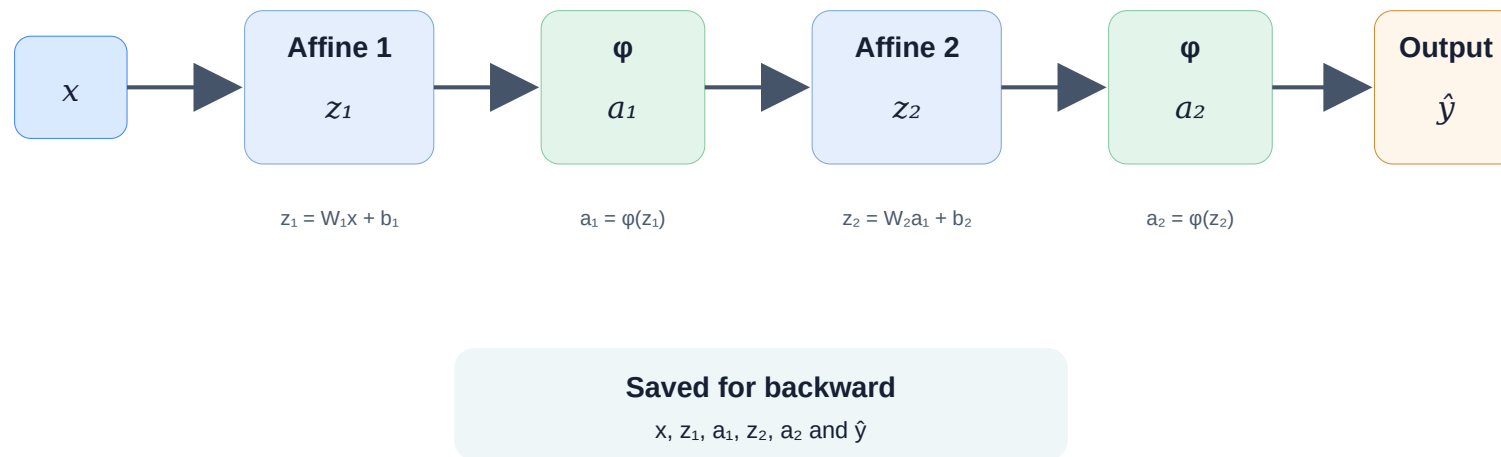
$$\nabla_{b_1} L = \delta_{z_1}$$

Backward propagates signals; each affine layer uses its signal to compute gradients for both W and b.

# Forward values and backward signals

## Forward pass through affine and nonlinear layers

Forward computes values left to right and stores intermediate values for backward.



[Open animation](#)

# Gradients for every parameter

Each trainable parameter receives:

$$\frac{\partial L}{\partial w_i}$$

The backward pass produces the information required by the optimizer.

# Backpropagation is not gradient descent

| Backpropagation                            | Optimizer                              |
|--------------------------------------------|----------------------------------------|
| computes $\nabla_{\theta} J_{\mathcal{B}}$ | uses $\nabla_{\theta} J_{\mathcal{B}}$ |
| differentiates the graph                   | updates the parameters                 |

backward  $\neq$  update

# Automatic differentiation

Manual view:

operations  $\rightarrow$  local derivatives  $\rightarrow$  chain rule

PyTorch:

```
loss.backward()
```

Autograd automates differentiation of the computational graph.

# The PyTorch training loop

```
optimizer.zero_grad()

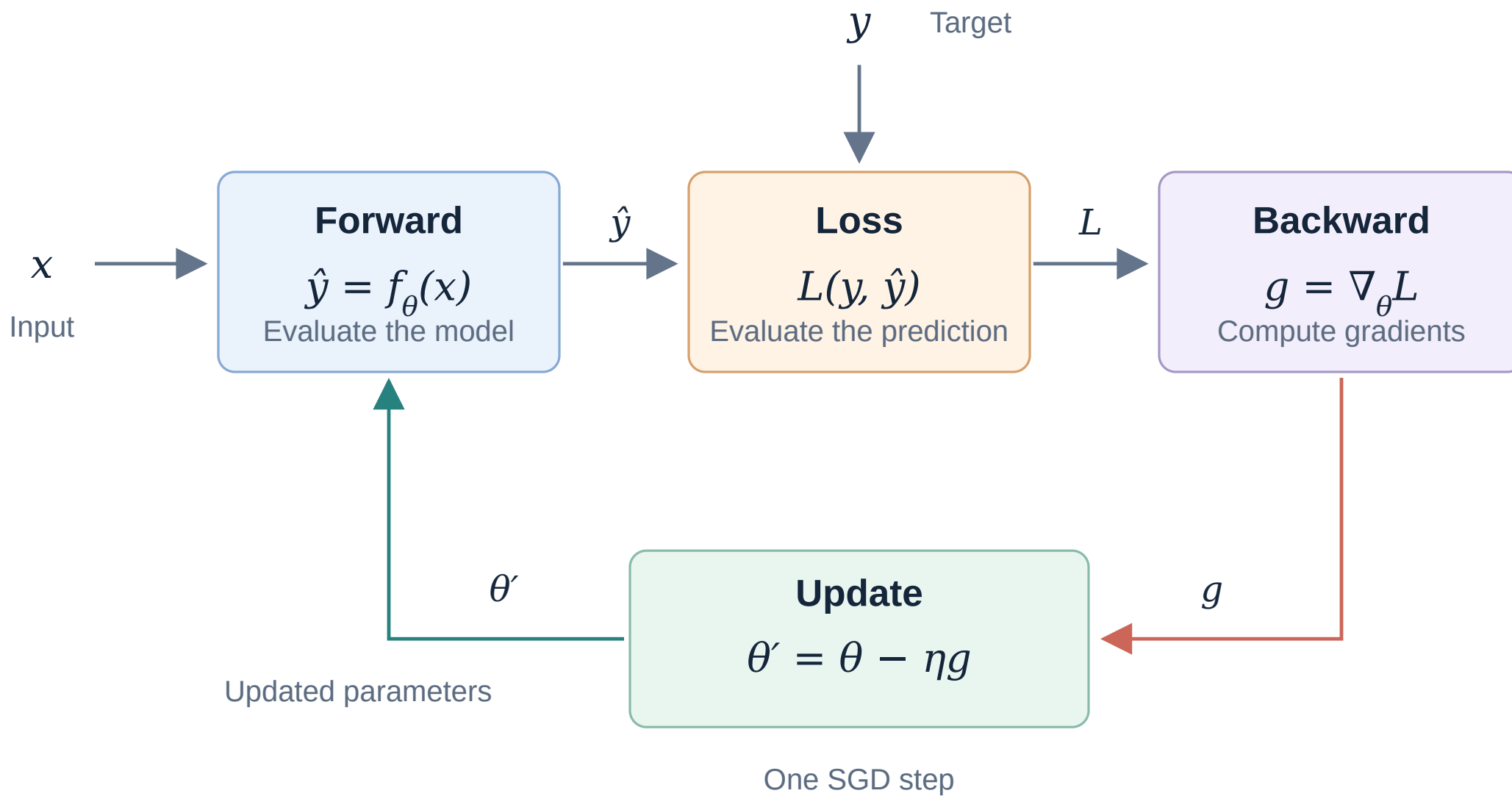
y_hat = model(x)
loss = criterion(y_hat, y)

loss.backward()
optimizer.step()
```

# From mathematics to PyTorch

Each training-loop operation has a direct implementation.

| Concept  | Mathematics                                             | PyTorch                            |
|----------|---------------------------------------------------------|------------------------------------|
| forward  | $\hat{y} = f_{\theta}(x)$                               | <code>y_hat = model(x)</code>      |
| loss     | $J_B(\theta) = \text{mean}_{i \in B} L(y_i, \hat{y}_i)$ | <code>loss = criterion(...)</code> |
| gradient | $\nabla_{\theta} J_B$                                   | <code>loss.backward()</code>       |
| update   | $\theta' = \theta - \eta \nabla_{\theta} J_B$           | <code>optimizer.step()</code>      |



# What changes in deep learning?

The learning loop remains the same.

What changes is:

- $f_{\theta}$
- architecture
- loss
- optimizer
- data
- scale

CNNs, RNNs, ViTs and Transformers do **not** replace the learning loop.

# Exit question

A student says:

“Backpropagation updates the weights to minimize the loss.”

What is incorrect in that statement?

# Guided practice — PyTorch in one hour

[Open notebook](#)

- tensors and affine transformations
- scalar autograd and parameter updates
- `TensorDataset` and `DataLoader`
- MLP for nonlinear classification
- loss curves and decision boundaries

Predict the behavior before running each experiment.

# Exercise — Multiclass MLP with tabular data

[Open notebook](#)

Dry Bean dataset from Kaggle: 16 attributes and 7 classes.

- split and standardize the data
- create a `DataLoader`
- implement an MLP and the training loop
- use `CrossEntropyLoss`
- evaluate accuracy, macro F1 and the confusion matrix

# Session 01 — Mental model

forward  $\rightarrow$  loss  $\rightarrow$  backward  $\rightarrow$  update

If this loop is clear, the rest of the course has a common foundation.